Øving 8: Traversering av grafer

Collapse context

Du må oppnå minst 13 av totalt 31 mulige poeng for å denne øvingen godkjent. Hver flervalgsoppgave er verdt 1 poeng, mens hver kodeoppgave er verdt 3 poeng. For informasjon om øvingsopplegget, se **nettsidene**. Trenger du hjelp, så **ta kontakt**.

Løsningsforslaget publiseres på **Canvas** etter innleveringsfristen.

Information

Author(s)	Ole Fredrik Borgundvåg Berg, Halvard Hummel, Daniel Hauksson
Deadline	10/22/26, 4:00:00 PM
Status	Not yet attempted
Grade	0.0%
Attempts	0
Submission limit	No limitation

Submitting as

Markus Granrud Arntzen

Group management

Scoreboard: Øving 8

Scoreboard: Alle øvinger

For evaluation

Best submission

No submission

Submission history

No submission

Question 1: Refleksjon

Dette er en refleksjonsoppgave. Denne oppgaven teller ikke mot antall oppgaver som må løses for å få godkjent øvingen. For mer informasjon om refleksjonsoppgavene, se **nettsidene**.

Refleksjonsoppgavene handler kun om refleksjon rundt forståelse av og arbeid med fagstoffet og øvingen. Pass på så du ikke inkluderer noen **sensitive personopplysninger** i refleksjonen din.

Før du begynner å gjøre øvingen, reflekter over følgende:

- Hvilke deler av pensum knyttet til denne øvingen er du usikker på?
- Hva tror du at du kommer til å lære ved å gjøre denne øvingen?

Etter du er ferdig med øvingen, reflekter over følgende:

- Hva forstår du nå, som du ikke forsto før du begynte å jobbe med øvingen?
- Hvilke nye sammenhenger ser du?
- Kan du komme på flere anvendelser av stoffet fra denne øvingen? I så fall hvilke?
- Er det noe du fortsatt er usikker på? I så fall hva?
- Hvis du skulle gjort øvingen på nytt, hva ville du gjort annerledes?

Lagger fortsatt her

Question 2: Nabolister

Hvilken av grafene er representert med følgende nabolister?

$1 \to \langle 3, 4 \rangle$

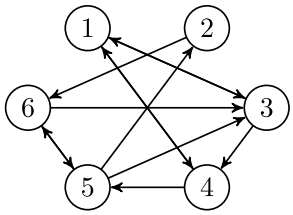
$2 \to \langle 6 \rangle$

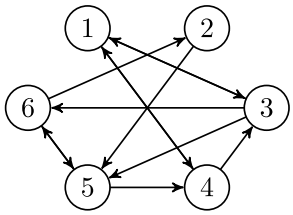
$3 \to \langle 1, 4 \rangle$

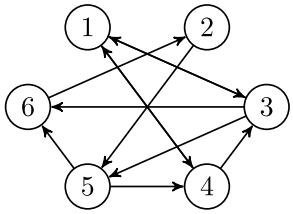
$4 \to \langle 1, 5 \rangle$

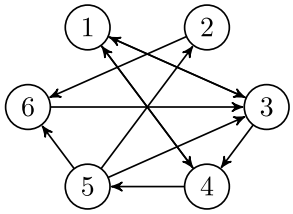
$5 \to \langle 2, 3, 6 \rangle$

$6 \to \langle 3, 5 \rangle$

- 

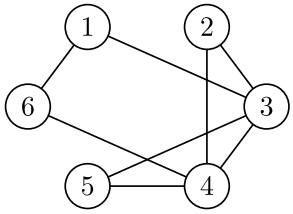
- 

- 

- 

Question 3: Nabomatriser

Hva er nabomatrisen til grafen under?



○
$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{array} \begin{array}{c} \begin{array}{cccccc} 1 & 2 & 3 & 4 & 5 & 6 \end{array} \\ \begin{bmatrix} 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \end{array}$$

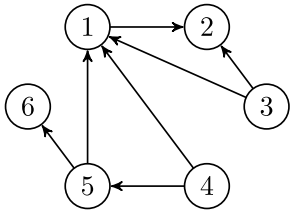
○
$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{array} \begin{array}{c} \begin{array}{cccccc} 1 & 2 & 3 & 4 & 5 & 6 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

○
$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{array} \begin{array}{c} \begin{array}{cccccc} 1 & 2 & 3 & 4 & 5 & 6 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

•
$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{array} \begin{array}{c} \begin{array}{cccccc} 1 & 2 & 3 & 4 & 5 & 6 \end{array} \\ \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \end{array}$$

Question 4: Topologisk sortering

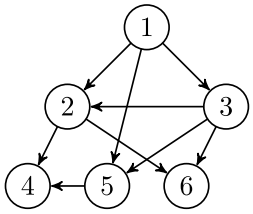
Hvilke sekvenser er topologiske sorteringer av denne grafen?



- ☐ $\langle 3, 1, 2, 4, 5, 6 \rangle$
- ☐ $\langle 4, 5, 6, 3, 2, 1 \rangle$
- ☐ $\langle 4, 5, 6, 1, 3, 2 \rangle$
- ☑ $\langle 4, 3, 5, 1, 6, 2 \rangle$
- ☑ $\langle 3, 4, 5, 6, 1, 2 \rangle$
- ☑ $\langle 3, 4, 5, 1, 2, 6 \rangle$
- ☐ $\langle 4, 5, 1, 6, 3, 2 \rangle$

Question 5: BFS

Du kjører BFS på denne grafen fra node 1:



Grafen er representert med nabolister, og rekkefølgen på kantene i nabolistene er tilfeldig. I hvilke rekkefølger er det mulig å besøke nodene i grafen?

- ☑ $\langle 1, 5, 3, 2, 4, 6 \rangle$
- ☐ $\langle 1, 3, 5, 2, 4, 6 \rangle$
- ☑ $\langle 1, 5, 2, 3, 6, 4 \rangle$
- ☑ $\langle 1, 2, 5, 3, 4, 6 \rangle$
- ☑ $\langle 1, 2, 5, 3, 6, 4 \rangle$
- ☑ $\langle 1, 3, 2, 5, 6, 4 \rangle$
- ☐ $\langle 1, 2, 3, 4, 5, 6 \rangle$

Question 6: Lage grafrepresentasjon

Et viktig problem knyttet til organdonasjon er å sikre at donoren og mottakeren av et organ er kompatible. En forenklet versjon av dette problemet, er at det finnes et antall biologiske og medisinske attributter som beskriver personene (f.eks. alder, kjønn og blodtype), og vi ønsker at donoren og mottakeren skal matche på flest mulig av disse attributtene. Det er da større sannsynlighet for en vellykket transplantasjon, fordi det ellers kan hende at immunforsvaret til mottakeren ødelegger det nye organet, en såkalt avstøtningsreaksjon. Av erfaring er det lav sannsynlighet for at kroppen avstøter seg et nytt organ dersom donor og mottaker matcher på minst k attributter.

Dette kan representeres som en graf, der det er en kant fra en donor til en mottaker dersom de har minst k attributter til felles. I forelesning 12 skal vi se at vi enkelt kan finne en matching mellom donorer og mottakere slik at flest mulig mottakere får en kompatibel donor hvis vi har en slik graf.

Du skal implementere funksjonen `compatibility_graph(donors, recipients, k)`. Her er `donors` og `recipients` lister av attributtene til henholdsvis donorer og mottakere: `donors[i][j]` er det j-te attributtet (representert ved en tekststreng) til den i-te donoren, og tilsvarende for `recipients[i][j]`. Listene har attributtene i samme rekkefølge, så en donor d og en mottaker m har et matchende attributt j dersom `donors[d][j] == recipients[m][j]`. Antallet attributter som må matche for at en donor og en mottaker skal anses som kompatible er gitt ved `k`.

Funksjonen skal returnere en liste `donor_edges`, der `donor_edges[i]` er en liste over alle mottakere (gitt ved deres indeks i `recipients`) i-te donor er kompatibel med. For eksempel, hvis input er `donors = [["ab"], ["ac"]]`, `recipients = [["ab"]]`, og `k = 1`, så skal funksjonen returnere `[[0], []]`. Dette er fordi den første donoren deler én attributt med den første mottakeren (og $1 \geqslant k$), mens den andre donoren ikke har noen felles attributter med noen mottakere.

Ønsker du å teste programmet ditt lokalt, finnes det eksempeltester her. Trenger du flere instanser å teste på lokalt, finnes det et større testsett som kan lastes ned.

```
1  def compatibility_graph(donors, recipients, k):
2      # Skriv din kode her
```

Question 7: Prioritetskøer i BFS og DFS

Hvilke typer prioritetskøer bruker vi i BFS og DFS til å holde styr på rekkefølgen vi skal besøke noder vi har sett, men ennå ikke besøkt?

- ○ Begge bruker en stakk.
- ○ BFS bruker en stakk, mens DFS bruker en kø.
- • BFS bruker en kø, mens DFS bruker en stakk.
- ○ Begge bruker en kø.

Question 8: Prioritetskøer og BFS

Hva gjør BFS hvis vi benytter en stakk som prioritetskøen i algoritmen?

- ○ Et dybde-først-søk av nodene som kan nås fra startnoden.
- • Et dybde-først-søk av alle nodene i grafen.
- ○ En topologisk sortering av nodene i grafen.
- ○ Et bredde-først-søk av alle nodene i grafen.
- ○ Et bredde-først-søk av nodene som kan nås fra startnoden.

Question 9: Pakkesystem

Har du noen gang trengt å installere pakker ved hjelp av et pakkesystem? En «pakke» er her programvare som installeres på datamaskinen. Python har for eksempel pakkesystemet pip som sin standard. Populære språkuavhengige pakkesystemer er APT, Homebrew, Pacman og Chocolatey.

En ting som gjør prosessen komplisert, er at en pakke kan avhenge av at andre pakker er installert for at den selv kan installeres. Disse andre pakkene kan igjen avhenge av at andre pakker er installert. På den måten kan man risikere å måtte installere veldig mange pakker. Heldigvis har vi ikke sykliske avhengigheter, siden det da ville ha vært umulig å installere pakken.

Her skal du implementere et forenklet pakkesystem. Du må implementere funksjonen `resolve_and_install(package)`, som er en funksjon som kalles når en bruker ønsker å installere en pakke. Her er `package` et `Package`-objekt som representerer en pakke klienten ønsker å installere. Et `Package`-objekt har attributtene `dependencies`, som er et tuppel med `Package`-objekter pakken avhenger av, og `is_installed`, som er en boolsk variabel som sier om pakken allerede er installert. Flere av pakkene kan allerede være installert før `resolve_and_install` kalles. Du kan anta at om en pakke er installert, så er alle pakkene den avhenger av også installert.

Funksjonen `resolve_and_install(package)` skal sørge for å installere pakken brukeren ønsker å installere. For å gjøre det, skal du bruke funksjonen `install`, som er implementert av oss. Når du kaller `install(package)` vil metoden prøve å installere pakken `package` på maskinen. Du må sørge for at følgende krav er overholdt når koden din kaller `install(package)`:

1. `package` kan ikke allerede være installert.
2. Alle pakkene som `package` er avhengig av må være installerte.

Ønsker du å teste programmet ditt lokalt, finnes det eksempeltester her.

```
1  def resolve_and_install(package):
2      # Skriv din kode her
```

Question 10: Grafrepresentasjoner - minnekompleksitet

Hva er minnekompleksiteten til en graf representert med henholdsvis *nabolister* og *nabomatrise*?

- ○ $\Theta(\mathrm{E})$ med nabolister og $\Theta(\mathrm{E}^2)$ med nabomatrise.
- ○ $\Theta(\mathrm{E})$ med nabolister og $\Theta(\mathrm{V}^2)$ med nabomatrise.
- ○ $\Theta(\mathrm{V}+\mathrm{E})$ med nabolister og $\Theta(\mathrm{E})$ med nabomatrise.
- • $\Theta(\mathrm{V}+\mathrm{E})$ med nabolister og $\Theta(\mathrm{V}^2)$ med nabomatrise.
- ○ $\Theta(\mathrm{E})$ med nabolister og $\Theta(\mathrm{E})$ med nabomatrise.

Question 11: Grafrepresentasjoner - tidskompleksitet

Du skal sjekke om det finnes en kant (u, v) i en graf. Hvor lang tid tar det, gitt at grafen er representert med henholdsvis *nabolister* og *nabomatrise*?

- • $\Theta(\mathrm{E})$ med nabolister og $\mathrm{O}(1)$ med nabomatrise.
- ○ $\Theta(\mathrm{E})$ med nabolister og $\Theta(\mathrm{V})$ med nabomatrise.
- ○ $\mathrm{O}(1)$ med nabolister og $\mathrm{O}(\mathrm{V})$ med nabomatrise.
- ○ $\mathrm{O}(\mathrm{V})$ med nabolister og $\Theta(\mathrm{V})$ med nabomatrise.
- ○ $\mathrm{O}(1)$ med nabolister og $\mathrm{O}(1)$ med nabomatrise.
- ○ $\mathrm{O}(\mathrm{V})$ med nabolister og $\mathrm{O}(1)$ med nabomatrise.

Question 12: Hvilken algoritme?

Du har et 3×3-spill som i figuren under, der hver rute er en brikke, bortsett fra den uten tall, som er tom:

2	3	6
5	8	
1	4	7

Du kan skyve en brikke som er over, under eller ved siden av den tomme ruten, inn i den tomme ruten. Målet er å få spillet på denne formen:

1	2	3
4	5	6
7	8	

Du vil finne ut hvordan man får til dette med færrest mulig flytt. Hvilken algoritme kan du bruke for å finne ut dette?

- ○ Topologisk sortering
- ○ DFS
- • BFS

Question 13: Hvilken algoritme?

Du har gått gjennom oversikten over emner på NTNU og laget en liste med emner du har lyst til å ta. Dessverre har noen av emnene i listen din andre emner i listen som forkunnskapskrav, som igjen har andre emner i listen som forkunnskapskrav, etc. – og flere av disse overlapper. Emnene har ingen emner utenfor listen som forkunnskapskrav, siden du sikret deg at du skulle ha alle nødvendige forkunnskaper da du laget listen. Du ønsker å finne en rekkefølge du kan ta emnene i, så du alltid tilfredsstiller forkunnskapskravene til et emne før du tar det. Hvilken algoritme passer best til å lage en slik rekkefølge?

- ○ Topologisk sortering
- ○ DFS
- ○ BFS

Question 14: Hvilken algoritme?

Denne oppgaven er basert på en eksamensoppgave.

Du er ansatt i et internasjonalt konglomerat. Konglomeratet har en veldig hierarkisk struktur, der hver ansatt har en sjef, der sjefen igjen har sin sjef, og slik går det helt opp til topplederen i konglomeratet. Enhver ansatt har akkurat én sjef, med unntak av topplederen, som ikke har noen sjef.

På grunn av størrelsen til konglomeratet, er det *veldig* mange nivå i hierarkiet, og det er derfor ønskelig å ha en datastruktur der tiden det tar å sjekke om en ansatt er underordnet en annen ansatt er $\mathrm{O}(1)$. En ansatt A er underordnet en annen ansatt B dersom B er sjefen til A, B er sjefen til sjefen til A, B er sjefen til sjefen til sjefen til A, osv.

Du har oppgitt de ansatte og deres sjefer i form av en graf, der nodene er ansatte, og en kant (u, v) betyr at u er sjefen til v. Det er mange som jobber i selskapet, så det er ønskelig at tiden det tar å lage datastrukturen er $\mathrm{O}(n)$, og at minnet datastrukturen krever er $\mathrm{O}(n)$, der n er antall ansatte. Anta at dersom du har en ansatt, er tiden det tar å finne noden tilhørende den ansatte $\mathrm{O}(1)$. Anta også at du kan velge hvilken node du starter å traversere fra i algoritmen du velger. Hvilken algoritme passer best for å lage denne datastrukturen?

- ○ Topologisk sortering
- ○ BFS
- ○ DFS

Question 15: BFS

Hva er sant om BFS?

- ☐ Man er garantert at alle nodene blir besøkt i traverseringen.
- ☐ Fungerer ikke om grafen har sykler.
- ☐ Kan brukes til å finne korteste vei fra en node til en annen dersom alle kantvektene er like og ikke-negative.

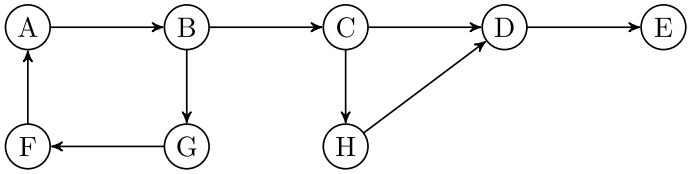
Question 16: DFS

Hva er sant om DFS?

- ☐ Kan brukes til kantklassifisering.
- ☐ Man er garantert at alle nodene blir besøkt i traverseringen.
- ☐ Fungerer ikke om grafen har sykler.
- ☐ Kan brukes til å finne korteste vei fra en node til en annen dersom alle kantvektene er like og ikke-negative.

Question 17: Sterkt sammenhengende komponenter

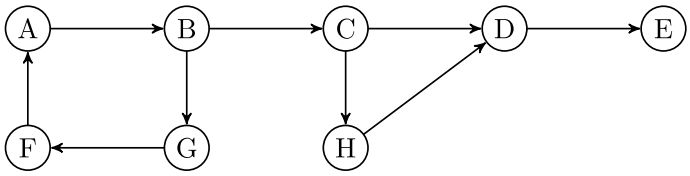
Hvis vi kaller STRONGLY-CONNECTED-COMPONENTS med grafen under som input og node A besøkes først, hvilke noder inngår i den første sterke komponenten som oppdages?



- [ ] B
- [ ] G
- [ ] A
- [ ] C
- [ ] H
- [ ] F
- [ ] E
- [ ] D

Question 18: Sterkt sammenhengende komponenter

Hvor mange sterke komponenter (*strongly connected components*) finnes det i grafen under?



- ○ 5
- ○ 1
- ○ 7
- ○ 3
- ○ 4
- ○ 2
- ○ 6
- ○ 8

Question 19: Sterkt sammenhengende komponenter

En rettet graf har $n > 1$ noder og like mange sterke komponenter. Hva vil skje med antallet sterke komponenter hvis vi legger til en kant?

- ○ Antallet vil aldri endre seg
- ○ Antallet vil minke med opptil $n - 1$
- ○ Antallet vil minke med 1
- ○ Antallet vil øke med 1

Question 20: Sterkt sammenhengende komponenter

Vi har en graf med $n > 1$ noder og m kanter. Hva stemmer om antallet sterke komponenter i denne grafen?

- ○ Hvis $m < n - 1$, vil antall sterke komponenter alltid være større enn 1.
- ○ Det finnes en konfigurasjon av noder og kanter som gjør at antall sterke komponenter er 0.
- ○ Hvis $m \geqslant n - 1$, vil antall sterke komponenter alltid være større enn 1.
- ○ Det finnes en konfigurasjon av noder og kanter som gjør at antall sterke komponenter er større enn n.

Question 21: Søppeltømming

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

En viktig del av minnehåndtering i datamaskiner er å oppdage og frigjøre biter med allokert minne som ikke lenger er i bruk.

For eksempel, i et Python-program kan man opprette variabler lokalt i en funksjon. Når funksjonen returnerer, så vil dataen i alle de lokale variablene fra funksjonen fortsatt ligge i minnet, men ikke lenger brukes videre i progammet. For å ikke gå tom for minne, må disse delene av minnet frigjøres, slik at de kan brukes på nytt i andre deler av programmet.

Prosessen som består av å oppdage og frigjøre allokert minne som ikke lenger er i bruk, kalles søppeltømming (*garbage collection*).

En ofte brukt metode for søppeltømming er tracing garbage collection. Her ser man på minnet som en samling av objekter med referanser (pekere) seg i mellom. Noen av disse objektene blir ansett som kjerneobjekter. Kjerneobjektene vil alltid være nødvendig å ha i minnet (f.eks. i løpet av hele levetiden til Python-programmet). Alle andre objekter i minnet klassifiseres som i bruk kun om de direkte eller indirekte er referert til fra minst et av kjerneobjektene. Metoden frigjør minne ved å utføre følgende to steg:

1. Finn og marker hvilke objekter i minnet som er i bruk.
2. Gå gjennom alle objektene i minnet og fjern dem som steg 1. ikke markerte som i bruk.

Beskriv en algoritme som, gitt alle kjerneobjektene, finner og markerer objektene i minnet som er i bruk.

Question 22: Rettferdig fordeling

I algoritmisk rettferdig fordeling ønsker man å finne en rettferdig fordeling av et sett med gjenstander til et sett med agenter, basert på deres preferanser. Dette kan for eksempel være fordeling av en arv (gjenstandene) til arvingene (agentene).

Når man skal fordele noe rettferdig, trenger man et rettferdighetsmål. Et naturlig krav, er at ingen agent skal være misunnelig på det en annen agent har fått (kalles envy-freeness). Dette kravet kan ikke alltid oppfylles, men man kan minimere mengden misunnelse, f.eks. ved å sjekke om det finnes en sykel med agenter som er misunnelige på hverandre. Da kan agentene bytte sine samlinger med gjenstander langs denne sykelen, og alle de involverte agentene får det bedre. Fjerning av slike misunnelsessykler spiller en viktig rolle i algoritmisk rettferdig fordeling.

> **Eksempel:** La agent 1 være misunnelig på agent 3, agent 3 være misunnelig på agent 2, og agent 2 være misunnelig på agent 1. Da blir alle agentene mer fornøyde, og vi har mindre misunnelse, om agent 1 får agent 3 sine gjenstander, agent 3 får agent 2 sine gjenstander og agent 2 får agent 1 sine gjenstander.

I denne oppgaven skal du implementere en funksjon som finner misunnelsessykler. Funksjonen skal ta inn antall agenter, `n`, og en matrise, `values`, bestående av tall. Verdien `values[i][j]` indikerer hvor mye agent `i` verdsetter agent `j` sin samling med gjenstander. Agent `i` er misunnelig på agent `j` dersom `values[i][j] > values[i][i]`.

Hvis det finnes en misunnelsessykel, skal funksjonen din returnere en tabell, `[a, b, c, ..., z]`, bestående av agenter (0-indeksert), slik at agent `a` er misunnelig på `b`, som igjen er misunnelig på `c`, etc., og agent `z` er misunnelig på `a`. Om det finnes flere misunnelsessykler, kan du returner en hvilken som helst av disse. Hvis det derimot ikke finnes noen, så skal metoden returnere `None`.

> **Eksempel:** Hvis input er `n = 3` og `values = [[1, 2, 2], [0, 1, 2], [0, 2, 1]]`, skal funksjonen din returnere en av de mulige svarene `[2, 1]` og `[1, 2]`. Da det er en misunnelsessykel hvor agent 1 er misunnelig på agent 2 og agent 2 på agent 1.

Hint 1 Hint 2

Ønsker du å teste programmet ditt lokalt, finnes det eksempeltester her.

```
1  def detect_envy_cycle(n, values):
2      # Skriv din kode her
```

Question 23: Kantklassifisering - tre

Du kjører kantklassifisering på et tre. Hvilke typer kanter kan du ende opp med?

- ☐ Foroverkanter (*forward edges*)
- ☐ Bakoverkanter (*back edges*)
- ☐ Krysskanter (*cross edges*)
- ☐ Tre-kanter (*tree edges*)

Question 24: Kantklassifisering - DAG

Du kjører kantklassifisering på en rettet asyklisk graf (dvs. en DAG). Hvilke typer kanter kan du ende opp med?

- ☐ Tre-kanter (*tree edges*)
- ☐ Bakoverkanter (*back edges*)
- ☐ Krysskanter (*cross edges*)
- ☐ Foroverkanter (*forward edges*)

Question 25: Veibygging

Som et steg i å forbedre infrastrukturen i et land, skal du bygge en vei som knytter samme to fattige landsbyer. Mellom landsbyene ligger det et fjellområde hvor byggekostnadene til tider kan være veldig høye. Veibyggingsprosjektet har et begrenset budsjett, og det er derfor deler av dette området veien ikke kan gå gjennom. For å holde kostnadene nede, og for å redusere reisetiden, ønsker du å finne ut hva den korteste veien du kan bygge er, gitt en oversikt over hvor det er for dyrt å bygge.

I denne oppgaven skal du implementere en funksjon `shortest_road(build_map, start, end)`. Her er `build_map` en tabell (en liste av lister) som representerer et rutenett av landskapet. For en posisjon (i, j) vil `build_map[i][j]` være `True` hvis det er mulig å bygge veien gjennom den gitte ruten og `False` ellers. `start` og `end` er tupler på formen `(i, j)`, og angir posisjonen til henholdsvis landsbyen veien skal starte i og landsbyen veien skal slutte i.

En liste av posisjoner danner en gyldig vei hvis den første posisjonen er `start`, den siste posisjonen er `end`, og man for enhver posisjon (i, j) i listen har at `build_map[i][j] == True`. I tillegg må posisjonene være slik at for enhver posisjon (i, j), er den neste posisjonen i listen en av de følgende posisjonene $\{(i-1, j), (i+1, j), (i, j-1), (i, j+1)\}$.

Funksjonen `shortest_road` skal returnere en liste med posisjoner på formen `(i, j)`, der

1. listen danner en gyldig vei, og
2. det ikke finnes en liste med færre posisjoner som gir en gyldig vei.

Dersom det ikke finnes en slik liste med posisjoner, skal funksjonen returnere `None`.

Ønsker du å teste programmet ditt lokalt, finnes det eksempeltester her.

Denne oppgaven teller mot poengtavlen til denne øvingen. For å kunne havne på poengtavlen må du løse oppgaven og sette variabelen `highscore` til `True` i koden din.

Rangeringen din er basert på hvor lang tid koden din bruker på å finne riktig svar.

```
1  def shortest_road(build_map, start, end):
2      # Skriv din kode her
3
4  # Sett 'highscore' til True hvis du vil vises på poengtavlen.
5  # For mer info se
   'https://algdat.idi.ntnu.no/ovinger.html#poengtavle'
6  # Merk: Kjøring for poengtavlen tar betraktelig lengre tid.
```

Question 26: Topologisk sortering med tabell

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

TOPOLOGICAL-SORT bruker en lenket liste til å holde orden på rekkefølgen til nodene.

Vi bytter ut denne lenkede listen med en dynamisk tabell (*dynamic table*). Vil dette fungere? Vil det endre kjøretiden til algoritmen? I så fall, finnes det noen måte å unngå dette på?

Question 27: Traversering - prioritetskø

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Denne oppgaven er et frempek i pensum og kan oppleves som vanskelig. Om du ikke får den til, fortvil ikke.

Køer og stakker er kun spesialiserte versjoner av en prioritetskø, hvor elemenetene prioriteres basert på når de ble med i køen/stakken. I køer får et element høyere prioritet jo tidligere det er satt inn i køen, mens det i stakker får høyere prioritet jo senere det er satt inn i stakken. Køer brukes gjerne i BFS, og stakker i DFS. Det er også mulig å bruke prioritetskøer med andre prioriteringer, noe som gjør at algoritmen får andre egenskaper.

Du har en vektet graf G og en node s. Du ønsker å finne lengden på korteste vei fra s til alle de andre nodene. Lengden på en vei vil her si summen av kantvektene langs veien.

Du prøver å traversere grafen fra s med en prioritetskø til å velge neste node, der prioieten er gitt av d-verdien til noden. Her er pseudokode:

```
TRAVERSE(G, s)
 1  let Q be an empty min-priority queue of vertices with their d-value as priority
 2  for each vertex u ∈ G.V
 3      u.d = ∞
 4  s.d = 0
 5  INSERT(Q, s)
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      for v ∈ G.Adj[u]
 9          if v.d = ∞
10              INSERT(Q, v)
11          v.d = min(v.d, u.d+weight of edge between u and v)
12          DECREASE-KEY(Q, v, v.d)
```

Vil $u.d$ være lengden på korteste vei fra s til u? Hvorfor/hvorfor ikke? Hva må vi eventuelt anta om grafen for at det skal være lengden på korteste vei?